SIMATS ENGINEERING

ASSESSMENT TOOL – C0-1 AT-1
COURSE NAME: Artificial Intelligence
COURSE CODE: CSA1735
NAME: Raghavan. K
Reg no: 192417003

Code of Conduct: I ____Raghavan. K/ 192417003__ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Raghavan. K

Experiment 2 Hospital Delivery Robot

Aim

To describe a hospital delivery robot using PEAS, classify its operating environment, and demonstrate a model-based utility-oriented delivery decision.

PEAS description

Element	Description
Performance measure	Correct and timely delivery; safe travel; no collision, loss, or sample mishandling; privacy; efficient battery use.
Environment	Wards, corridors, pharmacy, laboratory, lifts, doors, patients, staff, visitors, and other robots.
Actuators	Drive motors, steering and brakes, lift interface, compartment locks, display or speaker, and a handling arm if fitted.
Sensors	Cameras, lidar or depth sensors, proximity sensors, wheel encoders, IMU, barcode or RFID reader, lift status, compartment sensors, and battery monitor.

Environment classification

Property	Classification and reason
Observability	Partially observable: the robot cannot see every corridor, person, or obstacle at once.
Outcome	Stochastic: people move unpredictably; lift waits, wheel slip, and sensor noise affect travel.
Task structure	Sequential: each movement affects the robot's position, route, and later delivery decisions.
Change over time	Dynamic: people, obstacles, and lift availability can change during operation.
State and actions	Mixed, mainly continuous: position, speed, and time vary continuously; floors and delivery states are discrete.
Number of agents	Multi-agent: people and other robots share the hospital and affect the robot's choices.

Agent architecture

A model-based, utility-based planning agent is suitable. It maintains an internal map and current state because the whole environment is not visible. It plans toward delivery goals and weighs urgency, safety, battery use, and travel time.

Algorithm

1. Read the delivery destination, package identifier, robot location, and sensor readings.
2. Update the map and estimate the robot's current state.
3. If an obstacle blocks the path, stop safely, update the map, and replan.
4. If the robot is at the destination, verify the recipient and complete the handoff.
5. Otherwise choose a safe route action with the best balance of delivery time, safety, and battery use; repeat until deliveryis complete.

Program

```
def decide(percept):     if
percept["obstacle"]:         return "stop
and replan"     if percept["location"] ==
percept["destination"]:
        return "verify recipient and
deliver"     if
percept["next_action"] == "use
lift":         if not
percept["lift_available"]:
return "wait safely and replan"
return "move: " +
percept["next_action"]

state = {
"location":
"Pharmacy",
   "destination": "Ward 5",
   "obstacle": False,
   "next_action": "east corridor",
   "lift_available":
True, }
print(decide(state))
state["obstacle"] =
True
```

```
print(decide(state))
state["obstacle"] =
False
state["location"] =
"Ward 5"
print(decide(state))
```

Output

```
move: east corridor
stop and replan verify
recipient and deliver
```

Result: The PEAS model, environment properties, and a safe delivery decision cycle are specified.

Experiment 7 Food Delivery Route Optimisation

Aim

To construct a delivery tour with Nearest Neighbour and improve it with Hill Climbing using 2-opt swaps.

Algorithm

1. Start at hub H and repeatedly visit the nearest unvisited location.
2. Return to H and calculate the tour distance.
3. Try each 2-opt swap by reversing a segment between two non-adjacent tour edges.
4. Keep a swap only if it shortens the tour; stop when no swap improves it.
5. For this distance-minimization problem, a local minimum is better than its immediate neighbours, even if a better tourexists elsewhere. A plateau has equal-scoring neighbours; a ridge requires a sequence of moves that may not each improve the score.
6. Simulated Annealing can accept a longer tour with probability e^(-delta/T), where delta is the added distance and T istemperature. Lowering T over time helps the search explore early and settle later.

Program

```
D = {
    "H": {"P": 10, "Q": 15, "R": 20, "S": 25},
    "P": {"H": 10, "Q": 35, "R": 25, "S": 30},
    "Q": {"H": 15, "P": 35, "R": 30, "S": 20},
    "R": {"H": 20, "P": 25, "Q": 30, "S": 15},
    "S": {"H": 25, "P": 30, "Q": 20,
"R": 15}, }

def cost(route):
    return sum(D[route[i]][route[(i + 1) %
len(route)]]              for i in
range(len(route)))

def nearest_neighbour(start, stops):
    route = [start]
unvisited = set(stops)
while unvisited:
```

```
        current = route[-1]         nxt =
min(unvisited, key=lambda x:
D[current][x])         route.append(nxt)
unvisited.remove(nxt)     return route

def
two_opt(route):
while True:
        old_cost = cost(route)
improved = False         for i in
range(1, len(route) - 1):
            for k in range(i + 1, len(route)):
candidate = (route[:i] + route[i:k + 1][::-1]
+ route[k + 1:])                 if cost(candidate) <
old_cost:
                    route =
candidate
improved = True
break             if improved:
break         if not improved:
return route

tour = two_opt(nearest_neighbour("H", ["P",
"Q", "R", "S"])) closed_tour = tour + ["H"]
print(" -> ".join(closed_tour))
print("Distance:", cost(tour), "km")
```

Output

```
H -> P -> R -> S -> Q -> H
Distance: 85 km
```

Result: The Nearest Neighbour tour is already a 2-opt local optimum; no tested 2-opt swap reduces its 85 km distance.

Experiment 12 Mars Rover

Aim

To identify the rover's percepts, operating environment, actions, performance measures, and a suitable autonomous agent architecture.

Percepts and operating environment

Percepts include camera images, terrain and hazard distance, rover position and heading, tilt and wheel slip, sample and instrument readings, temperature, dust, battery or solar power, and communication status. The environment is partially observable, stochastic, sequential, slowly dynamic, and continuous. Local rover autonomy is usually single-agent; the wider mission with ground control is cooperative.

Actions and performance

Actions include driving, turning, stopping, selecting a route, imaging, taking measurements, positioning tools, collecting or analyzing samples, transmitting data, and conserving or recharging power. Performance is measured by scientific value and data quality, safe navigation, mission progress, successful data transmission, efficient energy and time use, and preservation of instruments and mobility.

Agent architecture

A model-based, utility-based planning agent with learning is suitable. The internal model supports decisions under partial observation and communication delay. Utility balances science goals, safety, time, and energy, while learning improves decisions from terrain and instrument experience.

Algorithm

1. Collect navigation, hazard, power, and science-instrument percepts.
2. Update the rover's world model and confidence in the observations.
3. If a hazard or unsafe tilt is detected, stop and replan.
4. If power is critically low, conserve energy and report status.
5. If a valuable sample is reachable safely, inspect or analyze it; otherwise drive to the next safe waypoint.
6. Store results, transmit when possible, and use observations to improve the model.

Program

```
def rover_action(p):     if
p["hazard"] or p["tilt_deg"] >
25:
        return "stop and
replan"     if
p["battery_pct"] < 15:
```

```
        return "conserve power and report"
 if p["sample_detected"] and
 p["science_score"] >= 7:
        return "analyze
 sample"     return "drive to
 next waypoint"

 percept = {     "hazard": False, "tilt_deg":
 8, "battery_pct": 62,
    "sample_detected": True,
 "science_score": 9, }
 print(rover_action(percept))
 percept["hazard"] = True
 print(rover_action(percept))
```

Output

```
analyze sample
stop and replan
```

Result: The rover senses, updates its model, selects a safe science-oriented action, and adapts when a hazard appears.

Experiment 14 OLA Cab Booking

Aim

To use a utility-based problem-solving agent to select a suitable cab for a trip while respecting the customer's explicit preferences.

Algorithm

1. Read the pickup location, destination, selected cab type, and comfort or budget preferences.
2. Retrieve available cabs and estimate each fare, pickup time, and route.
3. Discard options that violate hard constraints such as a selected cab type, maximum fare, or minimum comfort.
4. Calculate a utility score for each remaining option using comfort, fare, pickup time, and availability.
5. Show the best option and book only after the customer confirms; monitor the trip until the destination is reached.

Program

```
def choose_ride(cabs, min_comfort, max_fare, max_eta, selected=None):
    eligible = []     for cab in cabs:
if selected and cab["type"] !=
selected:
            continue         if
cab["comfort"] <
min_comfort:
            continue         if cab["fare"] >
max_fare or cab["eta"] > max_eta:
            continue         cab["utility"] = (10 *
cab["comfort"]                          - 0.05 * cab["fare"]
- 0.5 * cab["eta"])         eligible.append(cab)     return
max(eligible, key=lambda c: c["utility"]) if eligible else
None

cabs = [     {"type": "Mini", "fare": 180,
"eta": 5, "comfort": 3},
    {"type": "Sedan", "fare": 260, "eta": 8,
"comfort": 5},     {"type": "Prime", "fare": 320,
"eta": 6, "comfort": 5}, ] ride = choose_ride(cabs,
min_comfort=4, max_fare=350, max_eta=10) if
ride:
    print(ride["type"], "INR",
ride["fare"],           "ETA",
ride["eta"], "min",           "utility",
ride["utility"]) else:
    print("No ride matches the preferences")
```

Output

```
Sedan INR 260 ETA 8 min utility 33.0
```

Result: The utility-based agent selects the highest-scoring cab among the options that satisfy the customer's constraints.

Experiment 15 Uniform Cost Search

Aim

To find the least-cost path from warehouse S to warehouse G using Uniform Cost Search on the directed weighted graph in the assessment.

Algorithm

1. Insert the start state S into a min-priority queue with path cost 0.
2. Remove the entry with the smallest accumulated cost. Ignore it if it is an outdated, more expensive entry.
3. If the removed state is G, return its path and cost; UCS tests the goal when it is removed from the queue.
4. For every outgoing edge, add its cost to the current path cost. Insert the new path if it is cheaper than the best known pathto that state.
5. Repeat until G is removed or the queue is empty.

Program

```
import heapq from
itertools import
count

graph = {     "S": [("A",
1), ("G", 12)],
    "A": [("B", 3), ("C", 1)],
    "B": [("D", 3)],
    "C": [("D", 1), ("G", 2)],
    "D": [("G", 3)],
    "G": [],
}

def uniform_cost_search(start, goal):
    order = count()     frontier = [(0,
next(order), start, [start])]     best =
{start: 0}     expanded = []

    while frontier:
        cost, _, node, path =
heapq.heappop(frontier)         if cost !=
best.get(node):
            continue
expanded.append(node)
if node == goal:
return path, cost, expanded
```

```
        for neighbour, edge_cost in
graph[node]:
            new_cost = cost + edge_cost            if
new_cost < best.get(neighbour, float("inf")):
                best[neighbour] = new_cost
heapq.heappush(                    frontier,
(new_cost, next(order), neighbour,
path + [neighbour])                )
    return None, float("inf"), expanded

path, total, expanded =
uniform_cost_search("S", "G")
print("Path:", " -> ".join(path))
print("Cost:", total)
print("Expanded:", ", ".join(expanded))
```

Output

Path: S -> A -> C -> G
Cost: 4
Expanded: S, A, C, D, B, G

Result: UCS returns S -> A -> C -> G with total cost 4. The direct edge S -> G costs 12, so it is more expensive.